endava

Internship Programme

Internship program 2026

Introduction to TDD and BDD.

Agenda

01
INTRODUCTION AND NOTABLE PRACTICES

02
TEST DRIVEN DEVELOPMENT

03
BEHAVIOR DRIVEN DEVELOPMENT

04
ACCEPTANCE TEST-DRIVEN DEVELOPMENT

05
SUMMARY

06
TDD AND BDD IN AUTOMATION

endava

Internship Programme

Introduction And Notable practices

AGILE SOFTWARE DEVELOPMENT PRACTICES

Agile software development is supported by several concrete practices, covering areas like requirements, design, modeling, coding, testing, planning, risk management, process, quality, etc.

- Test Driven Development (TDD)
- Behavior Driven Development (BDD)
- Acceptance Test-Driven Development (ATDD)
- Design Driven Development
- Domain Driven Design (DDD)
- Pair Programming
- Continuous Integration (CI)
- Iterative and Incremental Development (IID)

OTHER APPROACHES

Approach	Description
Design Driven Development	It uses design as part of a process to learn and better define requirements in order to build better, more informed technology solutions. It can also be looked at as a process whereby design and user experience drives the development of a product or software application. This leads to products that people enjoy using and want to tell others about
Domain Driven Design (DDD)	An approach to software development for complex needs by connecting the implementation to an evolving model
Pair Programming	An agile software development technique in which two programmers work together at one workstation. One, the driver, writes code while the other, the observer or navigator, reviews each line of code as it is typed in. The two programmers switch roles frequently.
Continuous Integration (CI)	Continuous Integration (CI) is a development practice where developers integrate code into a shared repository frequently, preferably several times a day. Each integration can then be verified by an automated build and automated tests. While automated testing is not strictly part of CI it is typically implied.
Iterative and Incremental Development (IID)	A method of software development that is modeled around a gradual increase in feature additions and a cyclical release and upgrade pattern

endava

Internship Programme

TEST DRIVEN DEVELOPMENT

TEST DRIVEN DEVELOPMENT (TDD)

Test-Driven Development is a testing methodology, or a programming practice implemented from a developer's perspective.

The main intention of this technique is to modify or write a fresh code only when the test fails. Hence it results in lesser duplication of test scripts.

This technique is largely popular in agile development ecosystems. In a TDD approach, automated test scripts are written before functional pieces of code.

Is the code valid?

TEST DRIVEN DEVELOPMENT (TDD)

Now, we must understand the fact that TDD does not talk about the testing that testers do. Rather this approach actually talks about the testing which the programmers do.

Yes!! You guessed it right!!

It's unit testing.

The test which is written first is not the test that the testers write,
but it is the unit test which the programmers write.

TEST DRIVEN DEVELOPMENT (TDD)

Methodology Steps

Based on the requirements specified in the documents, a developer writes an automated test case

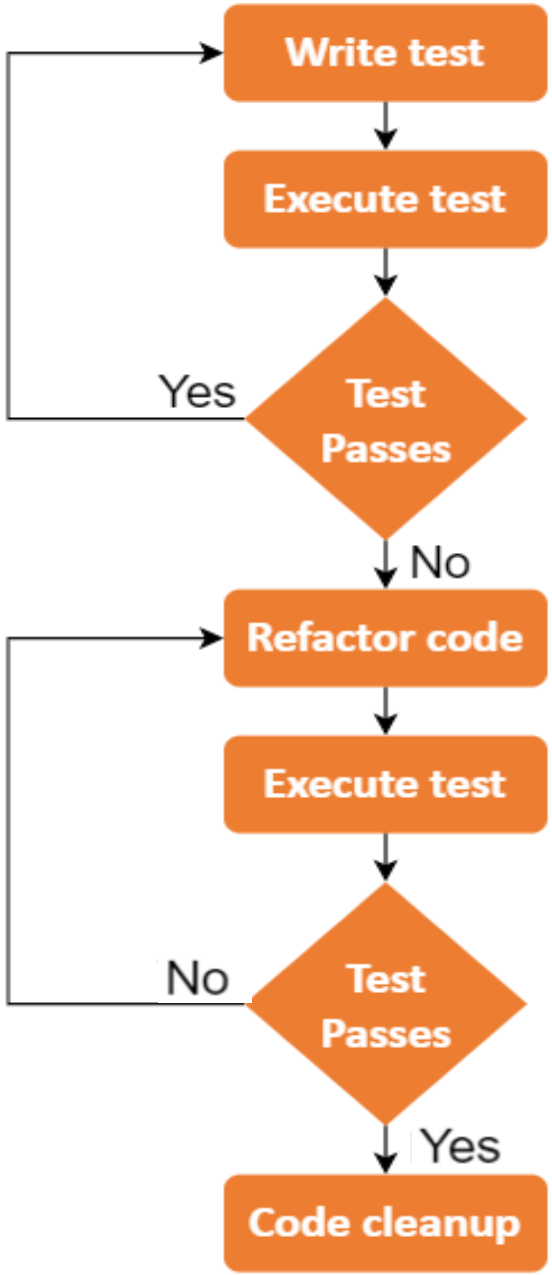
These tests are executed, and in some cases, they fail as they are developed before the development of an actual feature

The development team then re-factors the code for the test to pass successfully

endava

Internship Programme

FLOWCHART



The UNIT test is written first

The UNIT test is executed – which will fail (for obvious reasons)

The code is written (or refactored) just to make the test pass

The UNIT test is executed again

If the test passes, move on to next test ELSE re-write/modify the code to make the test pass

endava

Internship Programme

Mini Workshop

endava

Internship Programme

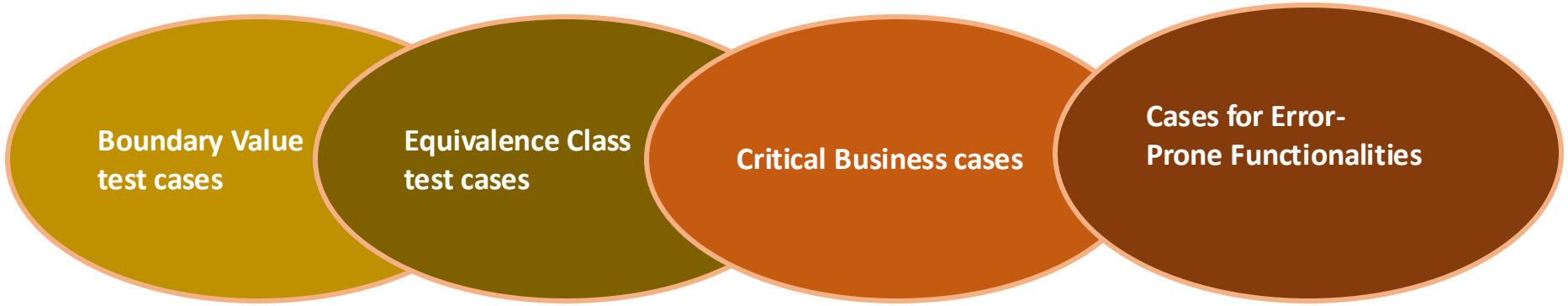
TEST DRIVEN DEVELOPMENT (TDD)

Now, the question that arises here is – if TDD is a programmer's job,

what is the tester's role in this approach?

Well, having said that TDD is a programmer's job, it does not mean that the testers are not involved in it.

In this technique, a QA engineer starts designing and writing test cases for every small functionality of an application. They can also collaborate by sharing the test scenarios consisting of:



BENEFITS OF TEST-DRIVEN DEVELOPMENT

1. Test coverage

2. Helps reduce the amount of time required for rework

3. Helps explore bugs or errors very quickly

4. Helps get faster feedback

5. Encourages the development of cleaner and better designs

6. Enhances the productivity of the programmer

7. Allows any team member to start working on the code in the absence of a specific team member. This encourages knowledge sharing and collaboration

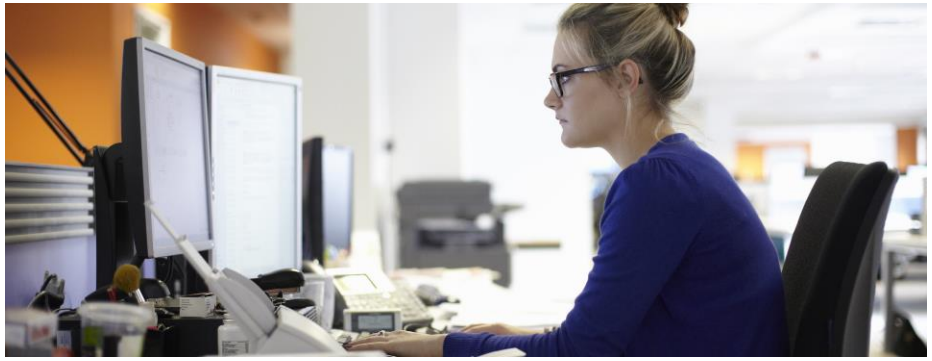
8. Gives the programmer confidence to change the large architecture of an application easily

9. Results in the creation of extensive code that is flexible and easy to maintain

DRAWBACKS TO TEST-DRIVEN DEVELOPMENT

1. It necessitates a lot of time and effort up front, which can make development feel slow to begin with.

2. Focusing on the simplest design now and not thinking ahead can mean major refactoring



3. It's difficult to write good tests that cover the essentials and avoid the superfluous

4. If the design is changing rapidly, you'll need to keep changing your tests. You could end up wasting a lot of time writing tests for features that are quickly dropped

5. The business owner should have a strong technical background and also be familiar with the unit test framework being used, which is not always the case

6. It takes time and effort to maintain the test suite – it must be reconfigured for maximum value

endava

Internship Programme

BEHAVIOUR DRIVEN DEVELOPMENT

Three Amigos

- Business – What's the problem?
- Development – How might we solve the problem?
- Testing – What could happen if?, What could go wrong?

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

Behavioral Driven Development (BDD) is a software development approach that has evolved from TDD (Test Driven Development).

TDD works satisfactorily, as long as the business owner is familiar with the unit test framework being used and their technical skills are strong enough, which is not always the case.
In these circumstances, BDD has the advantage because the test cases can be written in a common language used by the stakeholders such as, for example, English.

It differs by being written in a shared language, which improves communication between tech and non-tech teams and stakeholders. In both development approaches, tests are written ahead of the code, but in BDD, tests are more user-focused and based on the system's behavior.

This approach is making it possible for collaboration between the technical and non-technical teams to run with improved efficiency.

Essentials to have in place before implementing BDD

Requirements should be converted into user stories that can define concrete examples.

Each example should be a valid user scenario, rather than a mere test case.

An understanding of the 'role-feature-reason' matrix and the 'given-when-then' formula.

Who, What, Why

As a **customer**, I want to **enter coupon code during checkout** so I can **enjoy a discount in purchasing.**

An awareness of the need to write 'the specification of the behavior of a feature' rather than 'the unit test written for a feature'.

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

The ‘Given-When-Then’ formula BDD example

This is the proposed template for writing BDD test cases for a user story, which can be defined as:

Given a certain scenario (context)
When an action takes place (event)
Then this should be the outcome. (outcome)

Gherkin is a Domain Specific Language for writing acceptance criteria. It helps you to describe business behavior without the need to go into detail of implementation.

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

Other useful gherkin keywords

- Feature
- And, But
- Scenario
- Scenario outline
- Examples
- Rule
- Background

```
Feature: eat

Scenario: eating apples
Given there are 12 apples
And they are ripe
When 5 apples are eaten
Then 7 apples should remain
```

```
Feature: eat

Scenario Outline: eating cucumbers
Given there are <start> cucumbers
And they are ripe
When <eat> cucumbers are eaten
Then <left> cucumbers should remain

Examples: | start | eat | left |
          | 12    | 5   | 7    |
          | 20    | 5   | 15   |
```

endava

Internship Programme

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

HOW TO IMPLEMENT BDD?

There are many tools available in the market for implementing BDD.

cucumber

FitNesse

specflow

Concordion

jbehave

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

HOW DOES BDD HELP IN SDLC?

Debugging the errors in the latter stages of the development life cycle often proves to be very expensive. In the majority of cases, ambiguity in understanding the requirements is the root cause behind this. One needs to ensure that all the development efforts remain aligned towards fulfilling pre-determined requirements.

BDD allows developers to do the above by:

- Allowing the requirements to be defined in a standard approach using simple English
- Providing several ways to illustrate real-world scenarios for understanding requirements
- Providing a platform that enables the tech and non-tech teams to collaborate and understand the requirements

endava

Internship Programme

BENEFITS OF BEHAVIOR-DRIVEN DEVELOPMENT

1. BDD Is a cost-effective technique

2. You are no longer defining ‘test’, rather defining ‘behavior’

3. The behavioral approach defines acceptance criteria prior to development

4. Helps reach a wider audience by the usage of non-technical language

5. Focuses on how the system should behave from the customer’s and the developer’s perspective

6. Better communication between developers, testers and product owners

7. Because BDD is explained using simple language, the learning curve will be much shorter

endava

Internship Programme

DRAWBACKS TO BEHAVIOR-DRIVEN DEVELOPMENT

1. To work in BDD, prior experience of TDD is required

2. BDD is incompatible with the waterfall approach

3. If the requirements are not properly specified, BDD may not be effective

4. Testers using BDD need to have sufficient technical skills

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

- Easy to read and understand
- Declare behavior not procedures
- Don't repeat core keywords
- Be consistent with the steps, specify third person (specify user)

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

Write a few positive and negative test cases using GHERKIN for the following scenario:

As a user
I want to be able to login to the Dominos website
and order a pizza with free home delivery.

GIVEN

the User is logged in the domino website

And selects the order pizza option

WHEN

They fill the order data

And click on the order button

THEN

The order is placed

And the user notified

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

✓ **Concrete**
Use actual data values in your examples, be specific.

✓ **Interesting**
Every example should have a reason for existing. Try and give one an intention revealing name.

✓ **Declarative**
Examples that declare what the user is trying to achieve are easier to understand and maintain than those that describe imperatively what the user does.

✓ **Ubiquitous**
Avoid using terms that are understood by some of the team members only. Avoid using technical terminology.

✓ **Essential**
Any data you use in an example should contribute to understand the behavior being illustrated.

✓ **Focused**
Don't try to illustrate more than one interaction in any example.

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

Imperative

```
- Feature: Subscribers see different articles based on their subscription
  level
-
- Scenario: Free subscribers see only the free articles
-   Given users with a free subscription can access "FreeArticle1" but not
  "PaidArticle1"
-   When I type "freeFrieda@example.com" in the email field
-   And I type "validPassword123" in the password field
-   And I press the "Submit" button
-   Then I see "FreeArticle1" on the home page
-   And I do not see "PaidArticle1" on the home page
-
- Scenario: Subscriber with a paid subscription can access "FreeArticle1" and
  "PaidArticle1"
-   Given I am on the login page
-   When I type "paidPattya@example.com" in the email field
-   And I type "validPassword123" in the password field
-   And I press the "Submit" button
-   Then I see "FreeArticle1" and "PaidArticle1" on the home page
```

Declarative

```
- Feature: Subscribers see different articles based on their
  subscription level
-
- Scenario: Free subscribers see only the free articles
-   Given Free Frieda has a free subscription
-   When Free Frieda logs in with her valid credentials
-   Then she sees a Free article
-
- Scenario: Subscriber with a paid subscription can access both free
  and paid articles
-   Given Paid Patty has a basic-level paid subscription
-   When Paid Patty logs in with her valid credentials
-   Then she sees a Free article and a Paid article
```

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

```
-  Feature: Google Searching
-
-    Scenario: Google Image search shows pictures
-      Given the user opens a web browser
-      And the user navigates to "https://www.google.com/"
-      When the user enters "panda" into the search bar
-      Then links related to "panda" are shown on the results page
-      When the user clicks on the "Images" link at the top of the results page
-      Then images related to "panda" are shown on the results page
```

```
-  Feature: Google Searching
-
-    Scenario: Search from the search bar
-      Given a web browser is at the Google home page
-      When the user enters "panda" into the search bar
-      Then links related to "panda" are shown on the results page
-
-    Scenario: Image search
-      Given Google search results for "panda" are shown
-      When the user clicks on the "Images" link at the top of the results page
-      Then images related to "panda" are shown on the results page
```

endava

Internship Programme

BEHAVIOR DRIVEN DEVELOPMENT (BDD)

- ✓ Describing actions using the personal pronoun (i.e. I send the message to the server).
- ✓ No clear separation between Given/When/Then.
- ✓ Excess use of parameters and scenario outlines.
- ✓ Testing several rules at the same time.
- ✓ Writing the scenario after you've written the code.
- ✓ BA/Product Owner creating scenarios in isolation.
- ✓ Scenario with either a bad name or no name at all.
- ✓ Pointless scenario descriptions.
- ✓ Writing scenarios without talking to business people.
- ✓ High-level and vague scenarios.

endava

Internship Programme

```
Feature: Log in with multiple users

  Scenario Outline: 'Log in' in FB and Add New Friend
    When I log in with following data:
      | Name    | Password    |
      | <name> | <password> |
    Given I logged into FB
    Then I see the Home page displayed ,that contains
  "Hello <name>" and updates related to <me>
    Given user is searching for his friend <friend>
    When FB is displaying searching results
    And <me> add <friend> to friends by clicking 'Add' button
  and sending a friend request to server
    Then <me> and <friend> are friends now on FB
    Examples:
      | name    | password | me      | friend          |
      | Martha | pass2!   | Martha | Volodea Slănină |
```

endava

Internship Programme

ACCEPTANCE TEST-DRIVEN DEVELOPMENT

ACCEPTANCE TEST-DRIVEN DEVELOPMENT (ATDD)

Unlike Test-Driven Development (TDD) — in which tests are created from a developer's perspective — ATDD involves writing automated tests from the user's perspective.

It also and encourages collaboration between a functional expert and the development team. This collaboration provides development teams with the knowledge they need to make sure the software meets the acceptance criteria.

When creating a new feature using ATDD, one or more acceptance-level tests are written before any working code for the feature itself.

ACCEPTANCE TEST-DRIVEN DEVELOPMENT (ATDD)

Acceptance Test-Driven Development is very similar to Behavior-Driven Development. However, a key difference between them is: BDD focuses more on the behavior of the feature, whereas ATDD focuses on capturing the accurate requirements.

In Acceptance Test-Driven Development (ATDD) technique, a single acceptance test is written from the user's perspective. It mainly focuses on satisfying the functional behavior of the system. This technique attempts to answer the question:

Is the code working as expected?

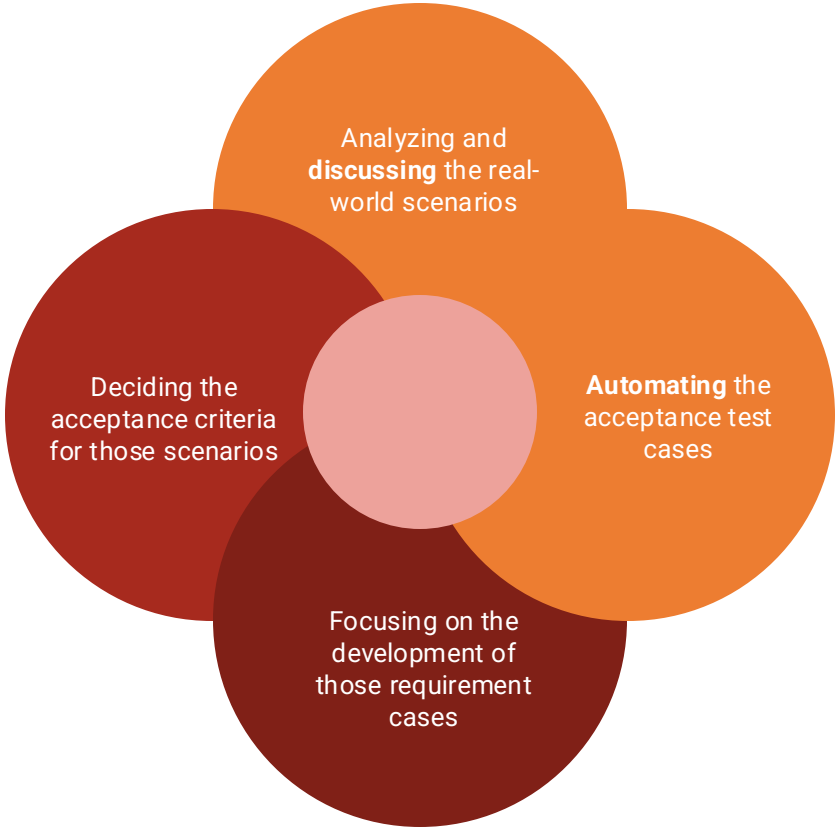
This technique enhances collaboration among developers, users, and QAs with a common focus on defining the acceptance criteria.

endava

Internship Programme

ACCEPTANCE TEST-DRIVEN DEVELOPMENT (ATDD)

KEY TECHNIQUES IN ATDD



BENEFITS OF ACCEPTANCE TEST-DRIVEN DEVELOPMENT

1. Requirements are very clearly analyzed without any ambiguity

2. Projects will often run more productively and result in a focused outcome that all stakeholders were expecting from the beginning

3. New features will be of higher quality, showing fewer defects and regression issues.

4. An ATDD framework also promotes better communication in teams and distributes responsibility so that each person can take ownership of the high-quality outcome.

5. It gives development teams a predefined target to aim for that keeps everyone aligned and on track

6. The acceptance test serves as a guide for the entire development process

endava

Internship Programme

DRAWBACKS TO ACCEPTANCE TEST-DRIVEN DEVELOPMENT

1. ATDD is less flexible because it becomes harder to change requirements after acceptance tests are written and coding has begun

2. If QAs fail to follow through, the implementation of this technique will lead to a huge number of high priority issues found on production

3. Acceptance tests aren't generally technical

4. If the test isn't working, it won't give you much insight into what has gone wrong

5. ATDD is also process heavy and may result in multiple tests with only minor differences

TESTING PYRAMID

1 **GUI Tests**

Selenium, Katalon, Test Complete

2 **Service- Level Tests**

BDD / ATDD

Fitnesse, Cucumber, Robot Framework

3 **Unit/Component Tests**

TDD

Xunit tools, JUnit, TestNG

endava

Internship Programme

Key Differences: TDD vs BDD vs ATDD

Parameters	TDD	BDD	ATDD
Definition	A development technique that focuses more on the implementation of a feature	A development technique that focuses on the system's behavior	A technique similar to BDD focusing more on capturing the requirements
Participants	Developer	Developers, Customer, QAs	Developers, Customers, QAs
Language used	Similar to the one used for feature development (Eg. Java, Python, etc)	Simple English, (Gherkin)	Simple English, Gherkin
Main Focus	Unit Tests	Understanding Requirements	Writing Acceptance Tests
Tools used	JDave, Cucumber, JBehave, Spec Flow, BeanSpec, Gherkin Concordian, FitNesse	Gherkin, Dave, Cucumber, JBehave, Spec Flow, BeanSpec, Concordian	TestNG, FitNesse, EasyB, Spectacular, Concordian, Thucydides

Homework

https://forms.gle/Yt4Nzib7C3Ueh56M8

Thank you!